Imena Vicky 26964 and kayiranga Deus 26699

Report of window queries

1. Compare Values with Previous or Next Records using LAG() and LEAD()

- **This query compares the salary of an employee with their previous and next records based on the hire date.**
    - **We use LAG() to get the previous salary, LEAD() to get the next salary, and a CASE statement to compare the salary with the previous one.**

```
SELECT
  id, name, salary,
  LAG(salary) OVER (ORDER BY hire_date) AS prev_salary,
  LEAD(salary) OVER (ORDER BY hire_date) AS next_salary,
  CASE
    WHEN LAG(salary) OVER (ORDER BY hire_date) IS NULL THEN 'NO PREVIOUS'
    WHEN salary > LAG(salary) OVER (ORDER BY hire_date) THEN 'HIGHER'
    WHEN salary < LAG(salary) OVER (ORDER BY hire_date) THEN 'LOWER'
    ELSE 'EQUAL'
  END AS compare_with_prev
FROM employee_analytics;
```

2. Ranking Data within a Category using RANK() and DENSE_RANK()

- **This query ranks employees within their department based on salary using both RANK() and DENSE_RANK().**
- **RANK() allows gaps between ranks if there are duplicates, while DENSE_RANK() does not.**

```
SELECT
  name, department, salary,
  RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank,
  DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS
dense_rank
FROM employee_analytics;
```

3. Identifying Top Records using RANK() for Top 3 Salaries per Department

- **This query returns the top 3 employees with the highest salary in each department.**
- **RANK() ensures that duplicate salaries are handled by skipping ranks.**

```
SELECT * FROM (
  SELECT
    id, name, department, salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
  FROM employee_analytics
)
WHERE rnk <= 3;
```

4. Finding the Earliest Records using ROW_NUMBER() (First 2 Employees per Department by Hire Date)

- **This query retrieves the first 2 employees who were hired in each department.**
- **ROW_NUMBER() ensures that we select the earliest employees without duplicates.**

```
SELECT * FROM (
  SELECT
    id, name, department, hire_date,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY hire_date ASC) AS
rn
  FROM employee_analytics
)
WHERE rn <= 2;
```

5. Aggregation with Window Functions

- **This query calculates the maximum sales per department (using PARTITION BY) and the overall maximum sales amount (using no PARTITION BY).**
- **It uses the MAX() window function to perform these aggregations.**

```
SELECT
```

```
  id, name, department, sales_amount,
  MAX(sales_amount) OVER (PARTITION BY department) AS max_per_dept,
  MAX(sales_amount) OVER () AS overall_max
FROM employee_analytics;
```